

EXPERIMENT 9

Write a Python Program to Solve the Travelling Salesman Problem (TSP)

Aim

To develop a Python program to solve the **Travelling Salesman Problem (TSP)** using the **Nearest Neighbour Algorithm** and determine the shortest possible route that visits every city exactly once and returns to the starting city.

Objective

- To understand the **Travelling Salesman Problem (TSP)**.
- To implement the **Nearest Neighbour Algorithm** in Python.
- To find an efficient route that visits all cities exactly once.
- To calculate the total distance travelled.

Theory

The **Travelling Salesman Problem (TSP)** is one of the most popular optimization problems in Artificial Intelligence. A salesman must visit a number of cities exactly once and return to the starting city while minimizing the total travelling distance.

Since checking every possible route becomes computationally expensive as the number of cities increases, heuristic algorithms such as the **Nearest Neighbour Algorithm** are commonly used to obtain a good solution in a shorter time.

The Nearest Neighbour Algorithm starts from a city and repeatedly selects the nearest unvisited city until all cities have been visited. Finally, it returns to the starting city.

Although it does not always guarantee the optimal solution, it is simple, efficient, and suitable for medium-sized problems.

Algorithm

Step 1: Define the distance matrix between all cities.

Step 2: Select a starting city.

Step 3: Mark the starting city as visited.

Step 4: Find the nearest unvisited city.

Step 5: Move to that city and add the distance to the total cost.

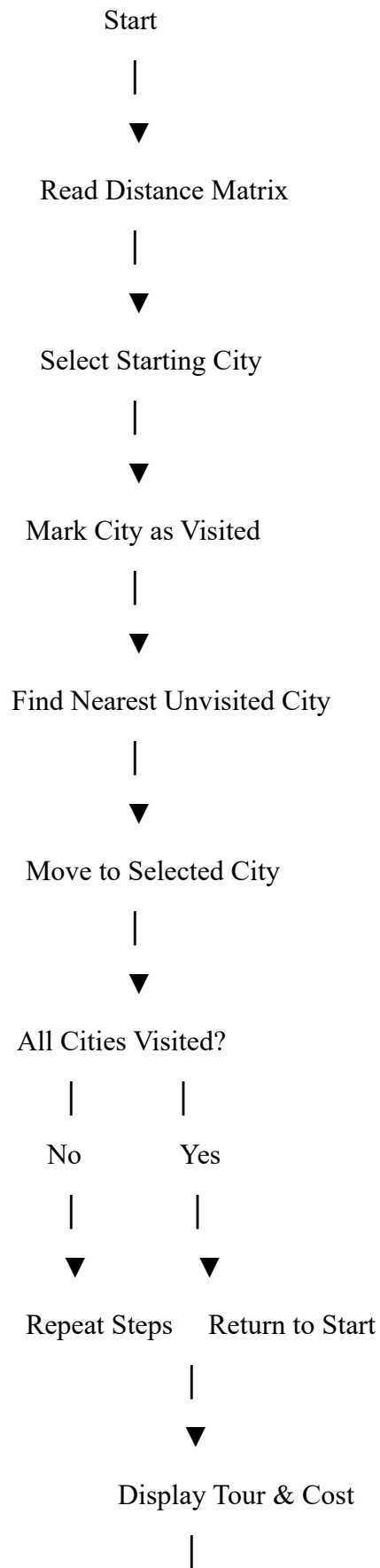
Step 6: Mark the new city as visited.

Step 7: Repeat Steps 4–6 until all cities are visited.

Step 8: Return to the starting city.

Step 9: Display the tour and the total distance travelled.

Flowchart





Stop

Python Program

```
def tsp(graph):  
  
    n = len(graph)  
  
    visited = [False] * n  
  
    current = 0  
  
    visited[current] = True  
  
    path = [current]  
  
    cost = 0  
  
    for i in range(n - 1):  
  
        minimum = float('inf')  
  
        next_city = -1  
  
        for j in range(n):  
  
            if not visited[j] and graph[current][j] < minimum:  
  
                minimum = graph[current][j]
```

```
next_city = j
```

```
visited[next_city] = True
```

```
path.append(next_city)
```

```
cost += minimum
```

```
current = next_city
```

```
cost += graph[current][0]
```

```
path.append(0)
```

```
return path, cost
```

```
graph = [  
    [0,10,15,20],  
    [10,0,35,25],  
    [15,35,0,30],  
    [20,25,30,0]  
]
```

```
path, cost = tsp(graph)
```

```
print("Tour :", path)
```

```
print("Minimum Cost :", cost)
```

Sample Output

Tour : [0, 1, 3, 2, 0]

Minimum Cost : 80

Explanation

Consider four cities represented as **0, 1, 2, and 3**.

The algorithm starts from **City 0**.

It chooses the nearest unvisited city at every step.

The tour obtained is:

0 → 1 → 3 → 2 → 0

The total travelling distance is **80**.

This route visits every city exactly once and finally returns to the starting city.

Advantages

- Simple and easy to implement.
- Produces a solution quickly.
- Suitable for medium-sized datasets.
- Reduces computation compared to exhaustive search.
- Useful for approximate optimization.

Disadvantages

- Does not always produce the optimal solution.
- Result depends on the starting city.
- May get trapped in a locally optimal route.

Applications

- Delivery route optimization.
- Logistics and transportation.
- Airline scheduling.
- Robot path planning.
- Circuit board manufacturing.
- Vehicle routing systems.

Result

The Python program successfully solved the **Travelling Salesman Problem (TSP)** using the **Nearest Neighbour Algorithm** and determined a valid route along with the total travelling cost.

Conclusion

The **Travelling Salesman Problem (TSP)** is a classical optimization problem in Artificial Intelligence. The **Nearest Neighbour Algorithm** provides a simple and efficient heuristic approach to obtain a near-optimal route by repeatedly selecting the closest unvisited city. Although it may not always produce the globally optimal solution, it significantly reduces computation time and is widely used in practical applications such as logistics, route planning, robotics, and transportation systems.

```

: > Users > Varshini M > OneDrive > Desktop > Here!!! > ARTIFICIAL INTELLIGENCE > VSC EXPs > EXP1.p
1  def tsp(graph):
2
3      n = len(graph)
4
5      visited = [False] * n
6
7      current = 0
8
9      visited[current] = True
10
11     path = [current]
12
13     cost = 0
14
15     for i in range(n - 1):
16
17         minimum = float('inf')
18
19         next_city = -1
20
21         for j in range(n):
22
23             if not visited[j] and graph[current][j] < minimum:
24
25                 minimum = graph[current][j]
26
27                 next_city = j
28
29         visited[next_city] = True
30
31         path.append(next_city)
32
33         cost += minimum
34
35         current = next_city
36
37     cost += graph[current][0]
38
39     path.append(0)
40
41     return path, cost
42
43
44     graph = [
45         [0,10,15,20],
46         [10,0,35,25],
47         [15,35,0,30],
48         [20,25,30,0]
49     ]
50
51     path, cost = tsp(graph)
52
53     print("Tour :", path)
54
55     print("Minimum Cost :", cost)
56

```

```

● PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft\python.exe" "c:/Users/Varshini M/OneDrive/Desktop
Tour : [0, 1, 3, 2, 0]
Minimum Cost : 80

```